

Separating KubeArmor's Observability Plane and Management Plane

A research-backed proposal to split KubeArmor's single, multiplexed gRPC server into two independently secured, scaled, and operated planes — grounded in the current codebase and validated against industry precedent from Kubernetes, etcd, Istio, Cilium/Hubble, SPIRE, Falco, and HashiCorp Vault.

STATUS

Draft for review

TYPE

Architecture design proposal

SCOPE

feeder / core / protobuf

AUDIENCE

Maintainers & infra engineers

PREPARED BY

Contributor Sagar Khandagre

Observability Server

LogService — high-volume streaming telemetry
(WatchMessages / WatchAlerts / WatchLogs)

Management Server

PolicyService, ProbeService, StateAgent — low-volume, high-trust administrative RPCs

Executive Summary

KubeArmor currently multiplexes four gRPC services — `LogService`, `PolicyService`, `ProbeService`, and `StateAgent` — onto a **single** `grpc.Server` **instance** (`fd.LogServer` in `KubeArmor/KubeArmor/feeder/feeder.go`), bound to a **single TCP listener**, secured by a **single TLS certificate**, and governed by a **single set of keepalive/interceptor settings**.

These four services are not peers. `LogService` is a high-volume, long-lived, read-only **telemetry stream** consumed by fan-out components such as KubeArmor-Relay and `karmor logs`. `PolicyService`, `ProbeService`, and `StateAgent` are low-volume, latency-sensitive, **administrative/control RPCs** consumed by orchestrators, operators, and human administrators. Coupling them at the transport layer means a telemetry storm can starve a policy update, a single certificate rotation affects both planes, and a single misconfigured interceptor can take down enforcement control alongside log delivery.

This proposal recommends splitting the daemon's gRPC surface into two independently-configured `grpc.Server` instances:

Observability Server

`LogService` only. Optimized for high fan-out, long-lived streaming, and external SIEM/log-pipeline exposure.

Management Server

`PolicyService`, `ProbeService`, `StateAgent`. Optimized for low-latency administrative operations, tightly scoped trust (Unix Domain Socket or `localhost` -only), and future authn/authz enforcement.

This mirrors an architectural pattern already validated at scale across the cloud-native ecosystem: Kubernetes (API server vs. kubelet read-only/secure ports), etcd (client port vs. peer port), Istio (Envoy data plane vs. Istiod control plane), Cilium/Hubble (embedded gRPC + Unix socket vs. Hubble Relay), SPIRE (Workload API over UDS vs. Server API over mTLS/TCP), and Falco (deprecating its own single embedded gRPC server model in favor of Unix-socket-first, decoupled output distribution). The remainder of this document analyzes the current design, justifies the split, and proposes a concrete target architecture.

Part 1 — Understanding the Existing Responsibilities

1.1 The Observability Plane (Data Plane)

Purpose. The observability plane exists to move a continuous stream of runtime security telemetry — system events, policy-violation alerts, and audit-style logs — from the point of collection (in-kernel eBPF/LSM hooks, translated by `KubeArmor/KubeArmor/monitor` and `KubeArmor/KubeArmor/feeder`) to the point of consumption

(log processors, SIEMs, humans). It is, in essence, **a telemetry pipeline that happens to be implemented as a gRPC service**, not a management API that happens to emit logs.

Components that belong to it, grounded in the current codebase:

- `LogService` (`KubeArmor/protobuf/kubearmor.proto`) — exposes `WatchMessages`, `WatchAlerts`, `WatchLogs`, all of which are **server-streaming RPCs**; the client opens one long-lived stream and receives events indefinitely.
- `feeder.BaseFeeder` / `feeder.Feeder` (`KubeArmor/KubeArmor/feeder/feeder.go`) — owns the `EventStructs` in-memory fan-out registry (`MsgStructs`, `AlertStructs`, `LogStructs`), each guarded by its own lock and pushed to via `PushMessage` / `PushLog` / `PushAlert`.
- `logServer.go` (`KubeArmor/KubeArmor/feeder/logServer.go`) — implements the per-client subscription lifecycle: `AddAlertStruct` / `AddLogStruct` register a bounded channel (`QueueSize: 1000`) per connected client, and the handler blocks on `svr.Send()` for the lifetime of the connection.
- **KubeArmor-Relay** — an external, independently-deployed component (see `pkg/KubeArmorOperator/common/defaults.go`, `KubeArmorRelayServerSecretName`, `GetRelayDeployment`) that connects to `LogService` as a client, aggregates logs/alerts across every node's `LogService` endpoint, and re-serves them to downstream consumers.
- `karmor logs` — the CLI client that either talks to a node's `LogService` directly (standalone/VM mode) or through the Relay (Kubernetes mode) to tail alerts/logs interactively.
- Future SIEM / log-pipeline adapters (e.g. Elasticsearch, Splunk, Fluent Bit sidecars) that need a stable, high-throughput, read-only feed.

Client characteristics. Observability clients are typically **long-lived, few-in-number-but-high-bandwidth, and read-only**. A single Relay connection may stay open for the lifetime of the pod and carry thousands of events per second during an incident. Clients never need to mutate state; they only `Watch*`.

Traffic characteristics.

- Predominantly **server push** over a small number of persistent HTTP/2 streams.
- **Bursty and adversary-correlated**: a container escape attempt, a crypto-miner, or a fork-bomb can generate thousands of alerts per second — precisely when the system is under the most stress and policy responsiveness matters most.
- Backpressure is already partially modeled today (`QueueSize: 1000`, drop/throttle logic in `AlertMap` / `AlertThrottleState`), which is itself evidence that this plane needs **its own** flow-control policy independent of anything else running on the same server.

Scalability requirements. Must scale with **event volume**, which is a function of workload count, policy verbosity, and attack surface — not with the number of administrative operations happening on the node. In large clusters (thousands of nodes, each streaming to a shared Relay tier) the observability plane is the dominant resource consumer of the KubeArmor gRPC surface by orders of magnitude.

Why it is fundamentally a telemetry pipeline. Every RPC in `LogService` is one-way, unbounded in duration, and stateless with respect to enforcement — the daemon's behavior (what gets blocked, what gets logged) never changes because a `WatchAlerts` client connected or disconnected. This is the defining property of a data-plane/telemetry component: it observes and exports; it does not decide.

1.2 The Management Plane (Control Plane)

Purpose. The management plane exists to let trusted operators and orchestration components **change what KubeArmor enforces** and **inspect its administrative/runtime state** — i.e., to drive the daemon's control loop, not to read its output.

Services that belong to it:

- `PolicyService` (`KubeArmor/protobuf/policy.proto`) — `containerPolicy` , `hostPolicy` , `networkPolicy` . Unary RPCs that push a serialized policy object and mutate `dm.SecurityPolicies` / `dm.Node` state (`KubeArmor/KubeArmor/core/kubeArmor.go` , `ParseAndUpdateContainerSecurityPolicy` and siblings). This is the non-Kubernetes (VM/bare-metal/systemd) analogue of what the Kubernetes CRD controller does via the API server in K8s mode.
- `ProbeService` (`KubeArmor/protobuf/policy.proto`) — `getProbeData` . A unary, on-demand introspection RPC (`KubeArmor/KubeArmor/core/karmorphobedata.go`) returning currently loaded containers, hosts, and policies — used by `karmor probe` for debugging/inspection, not for continuous consumption.
- `StateAgent` (`KubeArmor/protobuf/state.proto`) — `WatchState` , `GetState` . Pushes structured state-change events (container/node lifecycle) — administrative/state-sync traffic consumed by external state trackers (dashboards, state-aware controllers), gated behind `cfg.GlobalCfg.StateAgent` and only active in non-K8s mode (`KubeArmor/KubeArmor/core/kubeArmor.go` , `InitStateAgent`).

Clients that consume it: the `karmor` CLI (policy push, probe, debug), the KubeArmorController/operator in orchestrated deployments, provisioning/automation scripts in VM mode, and (in the future) any external policy-management or GitOps system reconciling desired policy state onto the node.

Why it is fundamentally different from telemetry. Management RPCs are **imperative and stateful**: calling `containerPolicy` changes what the enforcer (LSM/eBPF) permits from that point forward. A dropped or delayed management RPC has a direct, immediate security consequence — a policy update that should have blocked a newly-scheduled workload didn't arrive in time. Management traffic is low-volume (bytes per call, calls per minute-to-hour, not per second), latency-sensitive in a *correctness* sense rather than a throughput sense, and requires a fundamentally different trust bar: only entities authorized to change enforcement behavior should be able to reach it at all.

Part 2 — Current Design Problems

The problem is not merely "telemetry and control share a TLS cert." The two planes are coupled across nearly every architectural dimension, and none of that coupling is intentional — it is an artifact of `dm.Logger.LogServer` being the only `grpc.Server` the daemon happens to construct.

Dimension	Observability LogService	Management Policy / Probe / State	Consequence of sharing one grpc.Server
Client responsibility	Passive consumer: read-only stream tailing	Active operator: mutates enforcement state	A read-only client and a state-mutating client are indistinguishable at the transport layer; least-privilege can't be expressed

Dimension	Observability LogService	Management Policy / Probe / State	Consequence of sharing one grpc.Server
Trust assumption	"May observe security events"	"May change what is enforced"	Identical trust is required to reach either — there is no way to grant one without the other
Traffic pattern	Few long-lived streams, continuous push	Many short-lived unary calls, bursty on policy change	Both share the same keepalive (<code>kaep</code> / <code>kasp</code>) and connection settings tuned for neither case specifically
Workload shape	I/O-bound fan-out to N subscribers	CPU/lock-bound: policy parsing, enforcer syscalls	A slow <code>WatchAlerts</code> consumer occupies goroutines/buffers that management RPCs indirectly compete for on the same server's accept loop
Latency requirement	Throughput-sensitive (don't drop events)	Correctness-sensitive (policy must land before the workload runs)	No independent queue sizing or scheduling priority between the two exists today
Availability requirement	Nice to lose a few seconds of stream during restart	Must always be reachable to push an emergency policy	One <code>Serve()</code> failure (<code>ServeLogFeeds</code>) takes down policy delivery too, since it's the same listener/server object
Scaling characteristic	Scales with event volume × downstream consumers	Scales with number of administrators/controllers, ~O(1) per node	Cannot size the log path without also touching the code path that serves policy pushes
Client lifecycle	Long-lived, reconnect-and-resume semantics	Short-lived, transactional, request/response	Reflection (<code>reflection.Register(dm.Logger.LogServer)</code>) exposes both surfaces to any client that can reach the port at all
Operational ownership	Observability/SIEM teams, log infra, SREs	Security/platform teams enforcing policy	Both teams must coordinate on the same certificate, port, firewall rule, and on-call rotation
Failure domain	Log storm / stuck subscriber / SIEM outage backpressures the stream	Malformed policy blob / slow enforcer syscall blocks a unary RPC	A failure/backpressure event in one is not isolated from the other's goroutine pool and listener socket
Resource utilization	High: per-stream buffers × N streams, high-cardinality marshaling	Low: single small policy blob per call	Cannot grow the log path's memory/goroutine budget without over-provisioning the rarely-used management path identically, or vice versa

Why this matters beyond security. It would be easy to reduce this to "the ports should be different for security reasons," but the deeper issue is that **KubeArmor is currently unable to reason about, size, or operate these two planes independently**, because the code has only one seam — `dm.Logger.LogServer` — through which every gRPC concern (TLS, keepalive, health, reflection, service registration) flows. Concretely, today:

- `fd.LogServer = grpc.NewServer(grpc.Creds(tlsCredentials), grpc.KeepaliveEnforcementPolicy(kaep), grpc.KeepaliveParams(kasp))` in `feeder.go` is constructed **once**, and `PolicyService` / `ProbeService` / `StateAgent` are registered onto that same object later in `kubeArmor.go` (`pb.RegisterPolicyServiceServer(dm.Logger.LogServer, policyService)`, etc.).
- TLS is all-or-nothing (`cfg.GlobalCfg.TLSEnabled`): you cannot require mTLS for policy pushes while leaving log streaming on a simpler trust model, or vice versa, without a second listener.
- `reflection.Register(dm.Logger.LogServer)` advertises every service — telemetry and control alike — to anything that can complete a TCP handshake with the port.
- There is exactly one `net.Listener` (`fd.Listener`, bound to `cfg.GlobalCfg.GRPC`), so there is no way to bind management to `localhost` / a Unix Domain Socket while still exposing observability externally (or the reverse) without a structural change.
- Health status is tracked per-service (`dm.SetHealthStatus(pb.PolicyService_ServiceDesc.ServiceName, ...)`) but is served by the same health endpoint as the log service, so a health probe cannot cheaply express "is management up" independently of "is the log pipe up."

These two logical planes are coupled purely because they happen to share a transport object that was originally created to serve logs — `PolicyService` / `ProbeService` / `StateAgent` were bolted onto it later (`pb.RegisterPolicyServiceServer(dm.Logger.LogServer, ...)`) as a convenience, not as a deliberate design decision.

Part 3 — Proposed Architecture

3.1 Current State

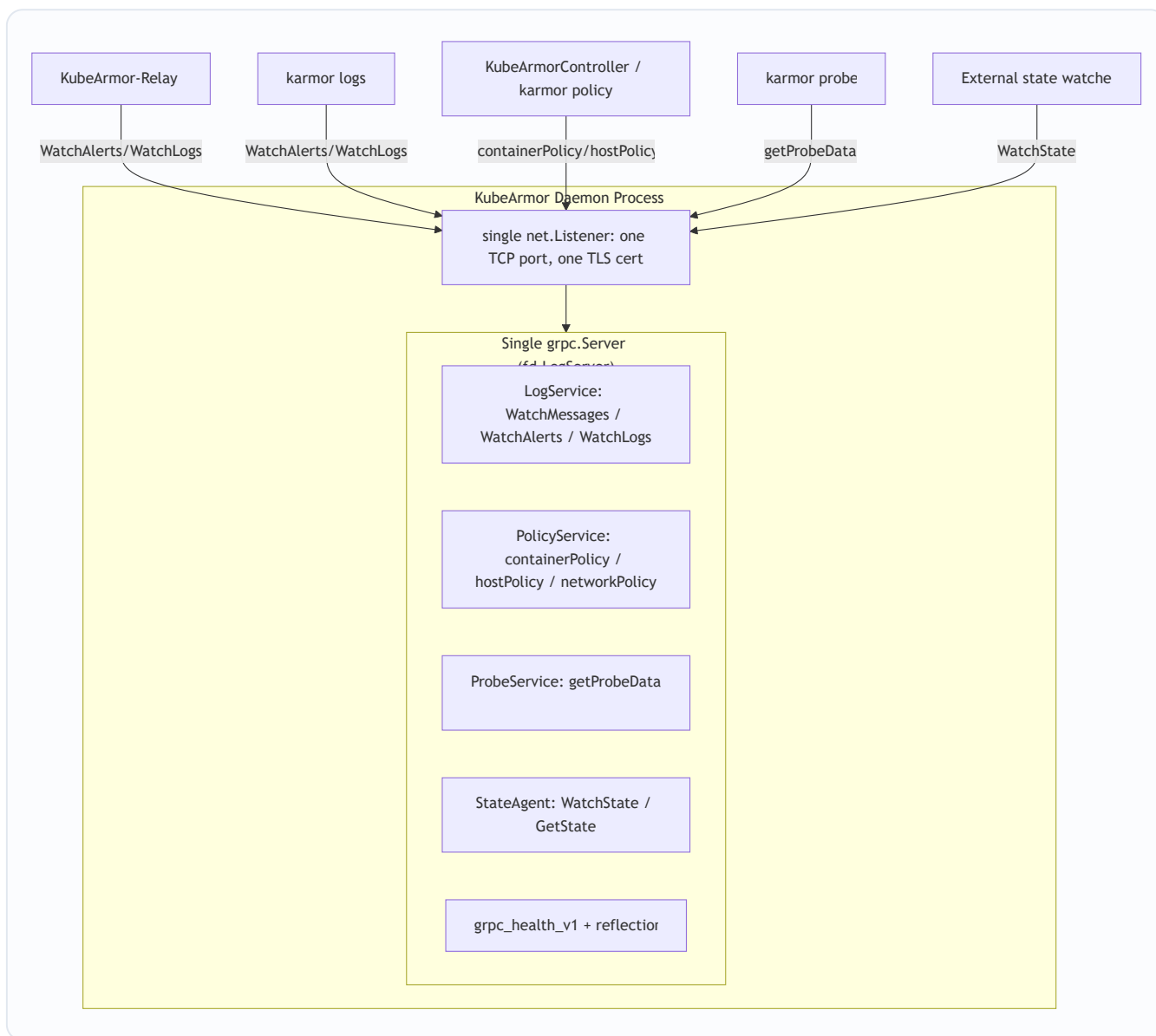


Figure 1 — Today: one `grpc.Server`, one listener, one certificate, four unrelated services.

Every consumer — regardless of trust level, traffic shape, or purpose — terminates on the same socket, the same certificate, and the same `grpc.Server` scheduling domain.

3.2 Proposed State

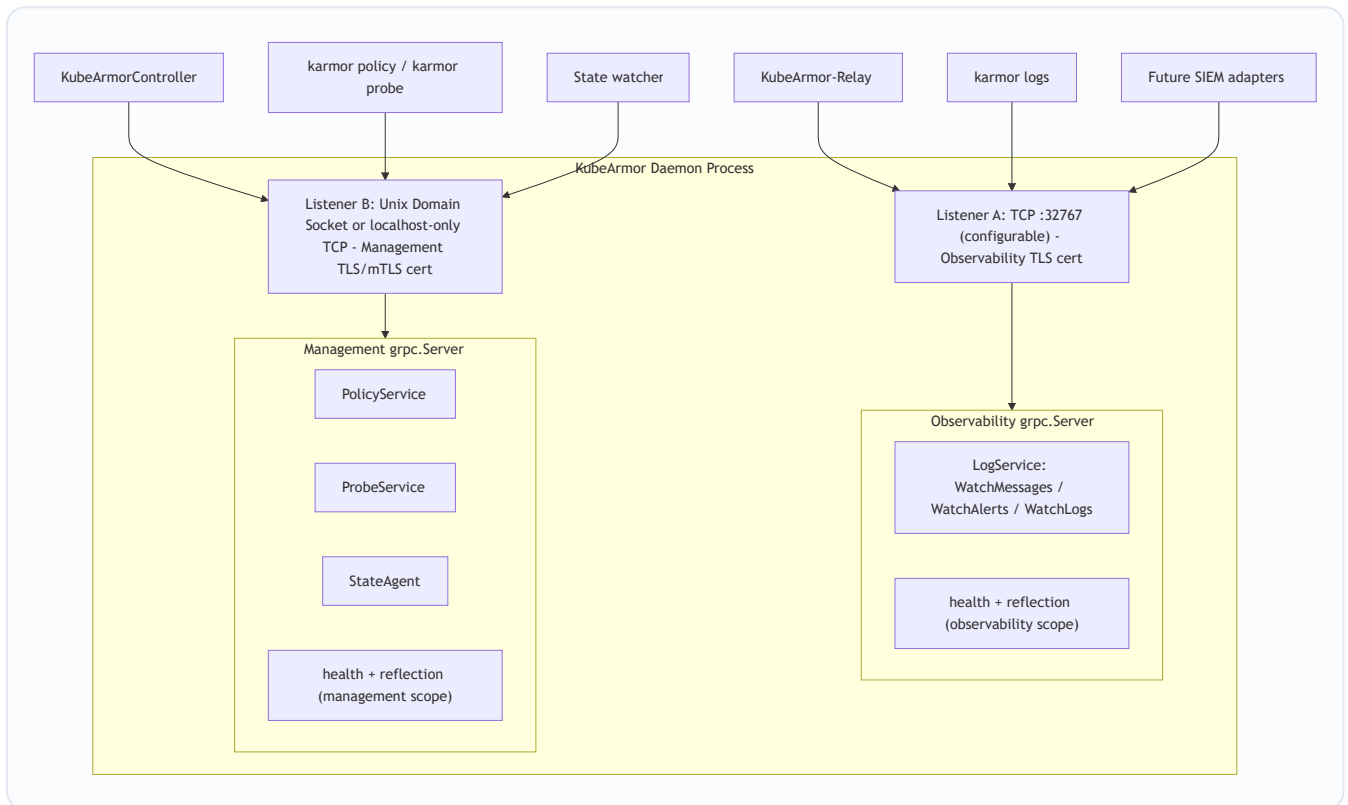


Figure 2 — Proposed: two independent `grpc.Server` instances, two listeners, two trust domains.

3.3 What changes structurally

- **Separate listeners.** Two `net.Listener`s replace `fd.Listener`: one for observability (kept on the existing configurable TCP port for backward compatibility), one for management (defaulting to a Unix Domain Socket under `/var/run/kubearmor/`, with an optional `localhost`-bound TCP fallback for environments without UDS support).
- **Separate `grpc.Server` instances.** `fd.LogServer` becomes `fd.ObservabilityServer`; a new `dm.ManagementServer` is constructed independently, each with its own `grpc.ServerOption` set.
- **Separate TLS configuration.** `loadTLSCredentials` is parameterized per-server: the observability server keeps the existing relay-facing certificate chain, while the management server can use a distinct, more tightly scoped certificate/trust bundle.
- **Separate interceptors/middleware.** Each server gets its own interceptor stack — rate limiting and audit logging on the management server, payload-size/backpressure tuning on the observability server.
- **Separate certificates and trust domains.** The management server's certificate can be scoped to a distinct SPIFFE-like trust domain/CA intended only for administrative clients.
- **Independent lifecycle.** `ObservabilityServer.GracefulStop()` and `ManagementServer.GracefulStop()` can be sequenced independently during shutdown/reload.
- **Independent deployment options.** An operator can expose only the observability port through a Kubernetes `Service` / `NetworkPolicy` while leaving the management port reachable only via a sidecar or Unix socket mount.
- **Transport flexibility.** The management server is a natural fit for a Unix Domain Socket (as Falco, Cilium/Hubble, and SPIRE already do), while the observability server remains TCP+TLS since its clients (Relay,

SIEM) are typically remote.

3.4 Sequence Diagrams

Today: policy push and log streaming interleave on one server

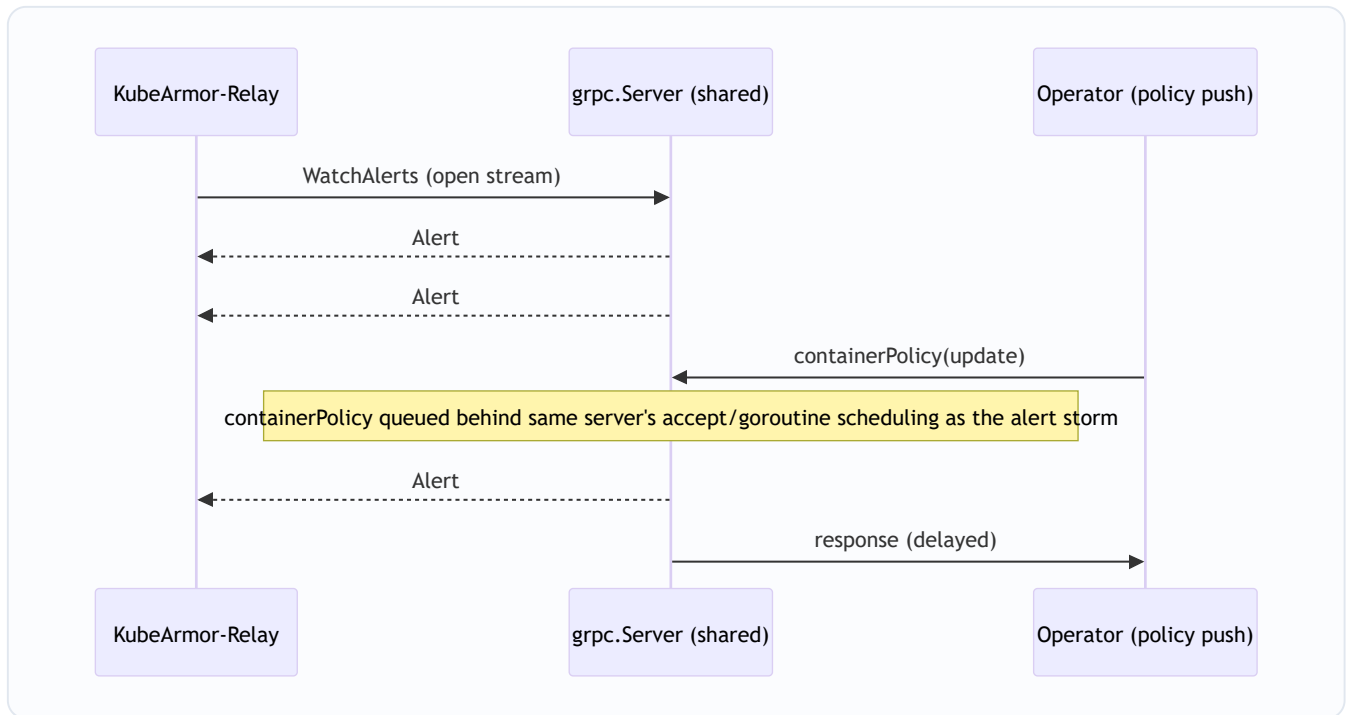


Figure 3 — A log storm on the shared server delays the response to a concurrent policy push.

Proposed: independent scheduling per plane

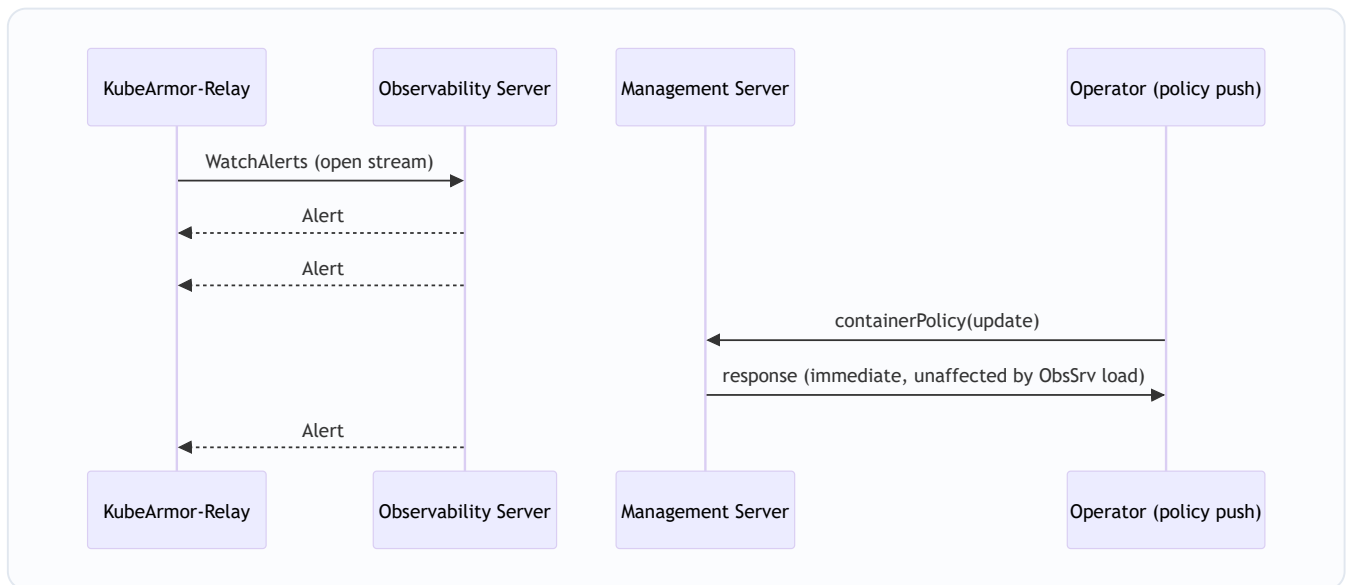


Figure 4 — With independent servers, the management response is unaffected by observability load.

Part 4 — Architecture Benefits

Security

- **Principle of least privilege.** A component that only needs to read logs never needs — and therefore never holds — credentials capable of pushing policy.
- **Trust domain separation.** Observability and management can be issued certificates from different intermediate CAs / SPIFFE trust domains.
- **Certificate separation** limits the blast radius of a leaked key: a leaked observability certificate lets an attacker read logs; it should never let them push a policy.
- **Independent authentication.** The management server can require mTLS or a bearer token unconditionally, even if the observability server allows a lighter-weight authentication mode.
- **Future authorization.** A per-RPC authorization interceptor is far easier to reason about when the interceptor chain is scoped to management RPCs only.
- **RBAC friendliness.** Kubernetes `NetworkPolicy` / `AuthorizationPolicy` controls map naturally onto "port A is observability, port B is management."
- **Easier audit.** Audit logging of "who changed policy" is a clean, low-volume log stream when isolated from telemetry.

Performance

- **Independent thread pools / goroutine scheduling** — a log fan-out storm no longer contends with the goroutines handling a `containerPolicy` unary call.
- **No starvation between telemetry and control** — the direct fix for the sequence-diagram scenario in §3.4.
- **Independent queue sizing.** The observability server's per-client buffers can be tuned for throughput without affecting the management server's footprint.
- **Independent backpressure.** A slow observability consumer can be throttled or disconnected without risk to the management path.
- **Separate resource allocation** — cgroup/goroutine budget planning per plane.
- **Better latency for policy operations**, no longer queued behind the observability server's scheduler.

Scalability

- **Independent scaling** — a per-node agent plus a separate management sidecar can scale on entirely different axes.
- **High-volume log streaming vs. low-volume management RPCs** no longer forced through identical connection/keepalive tuning.
- **Better horizontal scaling** of downstream consumers: Relay replicas scale with event volume, decoupled from operator activity.

Reliability

- **Log storms cannot delay policy updates** (and, symmetrically, policy churn cannot delay alert delivery).
- **Telemetry failures cannot impact management** — a `LogService` panic/restart no longer needs to restart `PolicyService`.

- **Management overload cannot impact telemetry** — a slow enforcer syscall does not block the accept loop serving `WatchAlerts`.
- **Better fault isolation and failure domains**, each independently observable and restartable.

Networking

- **Bind management only to `localhost` or a Unix Domain Socket**, drastically shrinking the network-reachable attack surface for policy mutation.
- **Independent ports** — no more overloading one port with reflection exposing both surfaces.
- **Different load balancers** — observability behind a streaming-aware LB; management avoids a shared LB entirely.
- **Different keepalive settings** tuned for very different connection-duration profiles.
- **Different connection limits**, so short management bursts can't exhaust a listener backlog sized for long-lived streams.

Rate Limiting

- **Independent rate limits and quotas** — aggressive per-IP limits on management RPCs without penalizing legitimate log streaming.
- **Separate DoS protection postures** for a more broadly-reachable observability endpoint vs. a tightly allowlisted management endpoint.

gRPC Features

- **Independent interceptors and middleware** scoped correctly to each plane.
- **Independent tracing/metrics/logging** — trace sampling for a high-rate log stream should differ from tracing for occasional policy pushes.
- **Independent reflection configuration** — safely enabled for observability, disabled/authenticated for management.

Operations

- **Easier maintenance** — each server reasoned about, load-tested, and documented independently.
- **Easier debugging** — a `karmor probe` hang is diagnosed entirely within the management server's logs/metrics.
- **Easier upgrades** — a protocol change to `LogService` carries zero risk to `PolicyService` wire compatibility.
- **Easier testing** — tests can spin up just the server relevant to the code path under test.

Future Extensibility

- **Future management authentication** (SPIFFE/mTLS, OIDC-derived identity) introduced on the management server alone.
- **Authorization interceptors** (e.g., OPA/Rego-based policy-push authorization) on a small, well-understood RPC surface.
- **Admission-style policies** — "reject this policy push unless signed by X" — become tractable as management-server interceptors.

- **Multi-tenant control plane** — a centralized policy-management service could speak only to the management server across a fleet.
- **External management APIs** — a REST/gRPC-gateway façade over just the management services becomes a contained, auditable surface.

Additional benefits discovered during analysis

- **Cleaner capacity planning.** A 1-second keepalive is far more appropriate for a long-lived streaming client than for a connect-once policy-push client; splitting lets each keepalive policy fit its traffic.
- **Cleaner health semantics.** Splitting makes "management is degraded but observability is healthy" a first-class, independently-probable signal for Kubernetes liveness/readiness.
- **Reduced blast radius for reflection-based enumeration attacks.** Splitting removes management service definitions from the observability-reachable surface entirely.

Part 5 — Industry Research: How Cloud-Native Systems Separate These Planes

Kubernetes — kubelet's read-only vs. secure port

Kubernetes historically shipped the kubelet with **two separate ports** for conceptually similar telemetry-vs-management data: `:10255`, an unauthenticated **read-only** port serving node/pod status and metrics, and `:10250`, an authenticated, authorized **secure** port serving the same and mutating operations. The read-only port was deliberately deprecated and disabled by default because an unauthenticated port — even one "just for reading" — became an unacceptable attack surface once the ecosystem matured. This is a direct precedent: read-mostly and authenticated/mutating concerns should not default to sharing a transport.

Reference: [Kubelet authentication/authorization](#); [Disable the kubelet read-only port \(GKE\)](#).

etcd — client port vs. peer port

etcd has shipped with **separate listeners** since its earliest versions: `--listen-client-urls` (default `2379`) for application/client gRPC traffic, and `--listen-peer-urls` (default `2380`) for Raft consensus traffic between cluster members. Notably, etcd's peer port *accidentally* accepted client-style traffic for years, and the community treated this as a bug: [PR #13565](#) explicitly removed the gRPC server from the peer listener, calling the leftover path "misleading" and something that "promotes bad usage patterns."

Reference: [etcd configuration options](#); [etcd-io/etcd#13565](#).

Istio — Envoy data plane vs. Istiod control plane

Istio is the canonical service-mesh example: the **data plane** (Envoy sidecars) carries and observes all application traffic, while the **control plane** (Istiod) handles configuration translation, service discovery, and certificate issuance. Istio's own security model documentation calls Istiod "a highly privileged component, similar to that of the Kubernetes API server itself," and treats its compromise as categorically more severe than a sidecar compromise.

Reference: [Istio Architecture](#); [Istio Security Model](#); [Istio component ports in detail](#).

Cilium / Hubble — the closest structural analogue to KubeArmor

Hubble's server is **embedded directly inside the `cilium-agent` process** (exactly as `LogService` is embedded inside the KubeArmor daemon) and exposes its Observer gRPC service **locally via a Unix Domain Socket, optionally also via TCP**. A separate, standalone **Hubble Relay** connects to every node's embedded Hubble server and re-serves a cluster-wide aggregated view — mapping almost one-to-one onto KubeArmor's own node-local `LogService` + external **KubeArmor-Relay** pattern.

Reference: [Hubble internals](#); [Cilium Component Overview](#).

SPIRE — Workload API (UDS) vs. Server API (mTLS/TCP)

SPIRE agents expose the **Workload API** exclusively over a **Unix Domain Socket**, authenticating callers via kernel peer credentials (`SO_PEERCRED`) rather than network-level TLS, because that API is deliberately scoped to local, same-host callers. The Agent-to-Server control channel, by contrast, runs over authenticated TCP with mTLS-derived trust, because it spans hosts.

Reference: [SPIRE Agent Configuration Reference](#); [SPIFFE Workload Endpoint spec](#).

Falco — the industry moving away from a single embedded gRPC server

Falco previously shipped exactly the shape KubeArmor has today: one embedded gRPC server exposing an "outputs" streaming API over a Unix socket or a TLS-secured TCP socket. As of Falco `0.43.0`, the project **deprecated its embedded gRPC output server entirely**, directing users toward decoupled, purpose-built output distribution (Falcosidekick) instead of a single server multiplexing everything.

Reference: [Falco gRPC Outputs docs \(deprecation notice\)](#); [falcosecurity/client-go](#).

HashiCorp Vault — multiple listener stanzas by design

Vault's configuration model treats "how many listeners, with what TLS and what purpose" as a first-class, repeatable configuration stanza, explicitly supporting per-listener TLS settings and even per-listener response redaction for unauthenticated endpoints. Vault separates the client-facing API address from the intra-cluster address, each with independent listener configuration.

Reference: [Vault TCP listener configuration](#); [Vault configuration parameters](#).

Synthesis

Across every system studied, the same shape recurs: **a high-volume, often-embedded, read/telemetry-oriented surface** is architecturally and operationally separated from **a low-volume, high-trust, mutation-oriented surface** — via independent listeners/ports/sockets, independent TLS material, and often independent processes altogether. None of the mature projects surveyed serve both concerns from one server as a long-term design; where that pattern existed historically (etcd's peer port, Falco's embedded gRPC server), it was treated as technical debt and removed.

Part 6 — Research at Scale

Large Kubernetes clusters / thousands of nodes. At fleet scale, the observability plane's aggregate load (N nodes × event rate) dwarfs the management plane's (N nodes × occasional policy push). A split model lets fleet

operators tune Relay-facing listener backlogs, buffer sizes, and connection limits purely as a function of telemetry volume, while the management listener remains minimally provisioned on every node — mirroring why Cilium ships Hubble embedded-but-locally-scoped with a dedicated Relay tier.

Standalone mode / edge deployments. KubeArmor's management plane is frequently operated by a human at the console or a local provisioning script — the co-located-caller pattern that SPIRE's Workload API and Falco's Unix-socket transport are optimized for. Binding management to a Unix Domain Socket by default removes the need for a management-plane TLS certificate at all on single-node edge deployments.

Multi-cluster deployments. A central policy-management or GitOps controller reconciling desired state across many clusters should only ever need to reach each node's management endpoint — never its (potentially far higher-volume) log endpoint.

Air-gapped deployments. Air-gapped environments frequently need to guarantee that no runtime security telemetry leaves the enclave except through an audited egress path, while still allowing local administrative tooling to manage policy freely inside the enclave. Two servers make this a natural default rather than an all-or-nothing network policy.

High-volume telemetry environments / enterprise SIEM integrations / SOCs. SOC-facing SIEM pipelines want a stable, high-throughput, purely additive feed and typically operate under a different change-management process than the team owning security policy. Splitting the transport gives SOC/SIEM engineering teams an interface they can consume, rate-limit, and monitor independently, without any risk to policy delivery latency.

Data Plane vs. Control Plane — Summary Comparison

Property	Observability LogService	Management Policy / Probe / State
RPC style	Server-streaming, long-lived	Unary (mostly), short-lived
Direction	Daemon → client (push)	Client → daemon (command), occasional daemon → client stream (<code>WatchState</code>)
Mutates enforcement state?	No	Yes (<code>PolicyService</code>), or reflects state (<code>StateAgent</code> , <code>ProbeService</code>)
Volume	High, bursty, adversary-correlated	Low, steady
Latency sensitivity	Throughput-sensitive	Correctness/urgency-sensitive
Ideal transport	TCP + TLS (remote fan-out)	Unix Domain Socket / localhost (local-first)
Ideal trust model	Read-only credential class	Mutating/admin credential class, mTLS-ready
Primary consumers	Relay, <code>karmor logs</code> , SIEM	<code>karmor policy</code> / <code>probe</code> , operator/controller
Failure impact	Delayed/lost telemetry	Delayed/failed enforcement change
Scaling axis	Event volume × subscriber count	Number of administrative actors (~O(1))

Property	Observability LogService	Management Policy / Probe / State
Analogous industry component	Envoy data plane, Hubble server, etcd client port	Istiod, SPIRE Server API, etcd peer port

Future Extensions

- 1. Authentication/authorization interceptors on the management server** — SPIFFE/mTLS-derived identity, or a simple static-token model as an interim step.
- 2. Per-RPC audit logging** on the management server, independent of telemetry volume.
- 3. Unix Domain Socket as the default management transport** on Linux, with TCP+TLS retained as an opt-in for remote management.
- 4. Rate limiting and quota interceptors**, tuned per plane.
- 5. Independent health/readiness endpoints** so Kubernetes probes can distinguish "telemetry degraded" from "management degraded."
- 6. A future gRPC-gateway/REST façade over the management server only.**
- 7. Multi-tenant / centralized control plane** speaking only to the management server across a fleet.

Conclusion

KubeArmor's `LogService`, `PolicyService`, `ProbeService`, and `StateAgent` are bound together today by an implementation detail — they all happen to be registered on `dm.Logger.LogServer` — not by any architectural necessity. They differ in traffic shape, trust requirements, latency sensitivity, failure domains, and operational ownership across every axis examined in this proposal. Splitting them into an **Observability Server** and a **Management Server** is consistent with how mature cloud-native systems — Kubernetes, etcd, Istio, Cilium/Hubble, SPIRE, Falco, and Vault — have independently converged on the same separation, and in at least two documented cases (etcd's peer port, Falco's embedded gRPC server), systems that once shared a transport for both concerns later treated that sharing as technical debt to be removed. This is a structural, incremental change: it does not require redesigning any RPC's semantics, only the transport and lifecycle plumbing that currently forces two fundamentally different planes through one seam.

References

- KubeArmor source: `KubeArmor/KubeArmor/feeder/feeder.go`, `KubeArmor/KubeArmor/feeder/logServer.go`, `KubeArmor/KubeArmor/core/kubeArmor.go`, `KubeArmor/KubeArmor/core/karmorprobedata.go`, `KubeArmor/protobuf/kubearmor.proto`, `KubeArmor/protobuf/policy.proto`, `KubeArmor/protobuf/state.proto`, `KubeArmor/pkg/KubeArmorOperator/common/defaults.go`.

- Kubernetes: [Kubelet authentication/authorization](#), [Disable the kubelet read-only port](#), [kubeadm PR #64187](#).
- etcd: [Configuration options](#), [etcd-io/etcd PR #13565](#), [etcd-io/etcd issue #11509](#).
- Istio: [Architecture](#), [Security Model](#), [Component ports in detail](#), [Pilot/Istiod architecture](#).
- Cilium/Hubble: [Hubble internals](#), [Cilium Component Overview](#).
- SPIRE: [SPIRE Agent Configuration Reference](#), [SPIFFE Workload Endpoint](#).
- Falco: [gRPC Outputs \(deprecation\)](#), [Unix socket for the gRPC server \(PR #1217\)](#), [falcosecurity/client-go](#).
- HashiCorp Vault: [TCP listener configuration](#), [Configuration parameters](#).